Database Systems

Self-Study Session
Advanced Topic:Blockchain

Yumeng Song
Department of Computer Science
Aalborg University
yumengs@cs.aau.dk
Fall 2025

Learning Objectives

- After completing this session, you will be able to:
  - Explain the origin, basic concepts, and classifications of blockchain systems.
  - Describe the layered architecture of blockchain and the role of each layer.
  - Understand how cryptographic hashes and consensus mechanisms maintain data integrity.
  - Discuss how smart contracts automate logic within the blockchain system.
  - Identify major real-world applications of blockchain across industries.

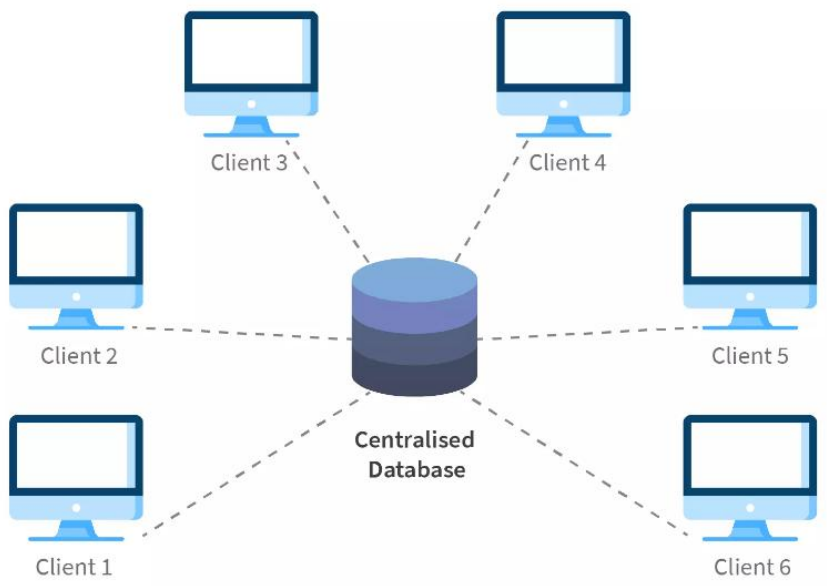
Agenda

- History and Motivation
- Basic Concepts
- Blockchain Architecture
  - Network Layer
  - Consensus Layer
  - Data Layer
  - Smart Contract Layer
  - Application Layer
- Blockchain Applications

Centralized Database Systems

- All previous discussions assume a centralized architecture, where one server or data center is responsible for data management and control.

  - Storage: All data tables, indexes, and logs are stored and maintained in a single location under unified administration.
  - Query Processing: Queries and updates are executed against one consistent copy of the database, ensuring deterministic results.
  - Transactions: Concurrent users operate on the same system, and coordination, locking, and recovery are all handled by the central server.



PostgreSQL, MySQL, Oracle, and SQLite are all based on this centralized database architecture.

Is Centralization Always Good?

- Centralized systems work well as long as the central authority remains trustworthy and competent.
- Advantages:
  - Simplified coordination and consistency
  - Efficient transaction management
  - Clear accountability and administration
- Limitations:
  - *Single point of failure*.
    - System integrity depends entirely on one authority or infrastructure.
  - *Limited transparency*.
    - Users cannot independently verify correctness or fairness.
  - *Power concentration*.
    - Misuse or corruption can compromise the entire system.
  - *Trust dependency*.
    - Users must believe that the central entity behaves correctly.

When Centralized Systems Lose Trust

- Zimbabwe’s Hyperinflation (2008–2009)
  - Zimbabwe had one of the highest inflation rates in recorded history: 231 million percent in July 2008.
  - In January 2009 the central bank printed a 100 trillion dollar note.
  - On the open market 10 trillion Zimbabwe dollars were worth about 30 US dollars.
  - People carried bags of cash for basic goods.

When Centralized Systems Lose Trust

- The Madoff Ponzi Scheme (2008)
  - In December 2008 Bernard Madoff, a Wall Street financier, was arrested for one of the largest investment frauds in history.
  - He caused investor losses of about 50 billion USD and was later sentenced to 150 years in prison.
  - The Ponzi scheme pays early investors with money from new investors while claiming steady returns.
  - Madoff's firm kept all records and statements under his sole control, hiding the lack of real investment activity.

When Centralized Systems Lose Trust

Broken trust

- Modern currencies depend on the credit of the state. Zimbabwe's hyperinflation was a collapse of national credit.
- Madoff's fraud showed how social reputation and authority can replace verification and bypass verification and avoid scrutiny.

Absolute centralization

- In Zimbabwe, all monetary control came from a single central bank.
- In Madoff's case, all financial information was controlled by one person.

Lack of transparency and oversight

- Zimbabwe's central bank issued currency without public auditing or disclosure.
- Madoff's fund operated with no external monitoring or visibility.

The Birth of Bitcoin

- Can technology replace authority as the source of trust?
  - In the 1990s a group known as the Cypherpunks began exploring this idea.
  - They argued that privacy and freedom online depend on cryptography, not on governments or corporations.

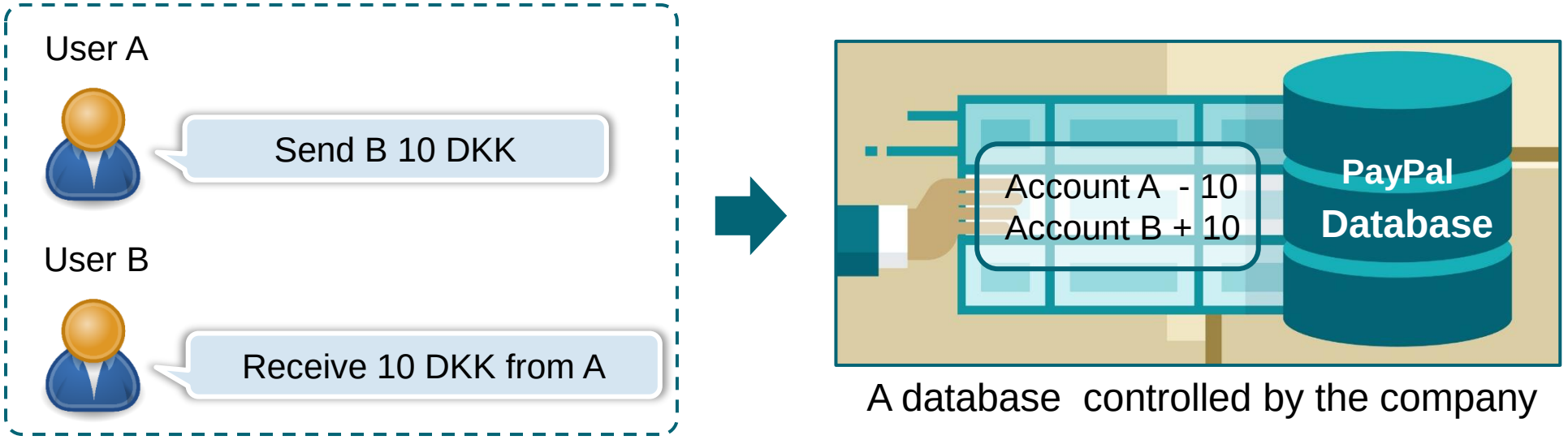
- In 2008, during the global financial crisis, someone using the name Satoshi Nakamoto published the paper

"Bitcoin: A Peer-to-Peer Electronic Cash System."

  - Proposed a decentralized cash system with no intermediaries.
  - Bitcoin launched as independent of central banks in 2009.
  - Satoshi mined the first block, the Genesis Block, and embedded message: "The Times 03/Jan/2009 Chancellor on brink of second bailout for banks."

Bitcoin

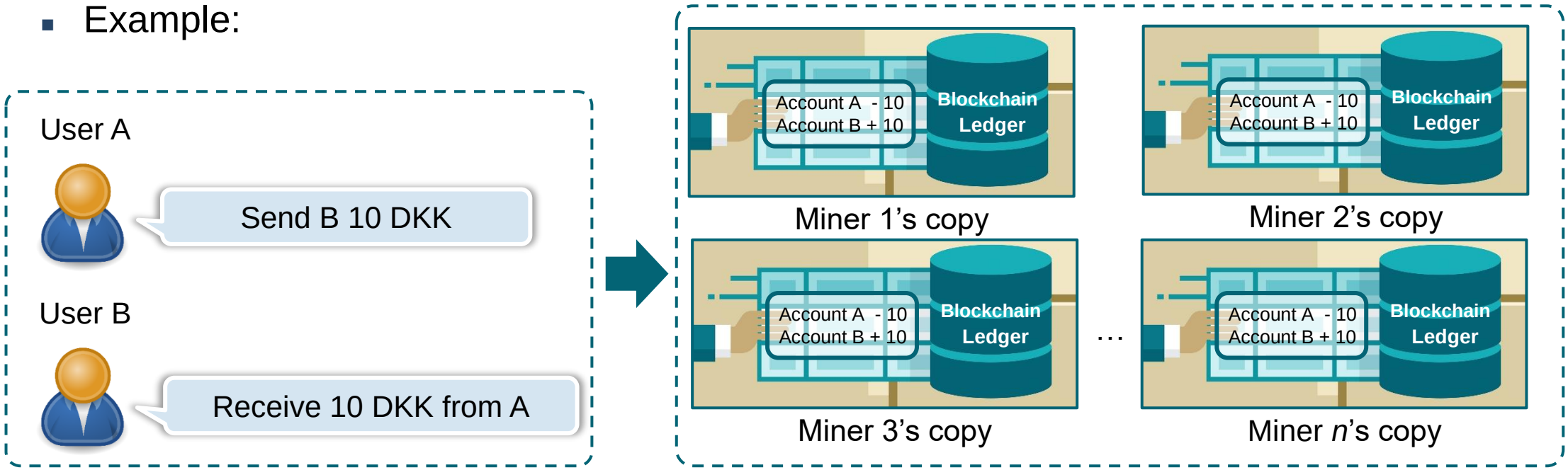
- Bitcoin can mean two things:
  - Bitcoin system: A peer-to-peer platform that records and verifies transactions.
  - Bitcoin (BTC): The digital currency traded on that platform.
- Bitcoin system works like MobilePay or PayPal, i.e., you can use it to send and receive value.
  - MobilePay and PayPal use one central database controlled by a company.
  - Example:



A database controlled by the company

Bitcoin

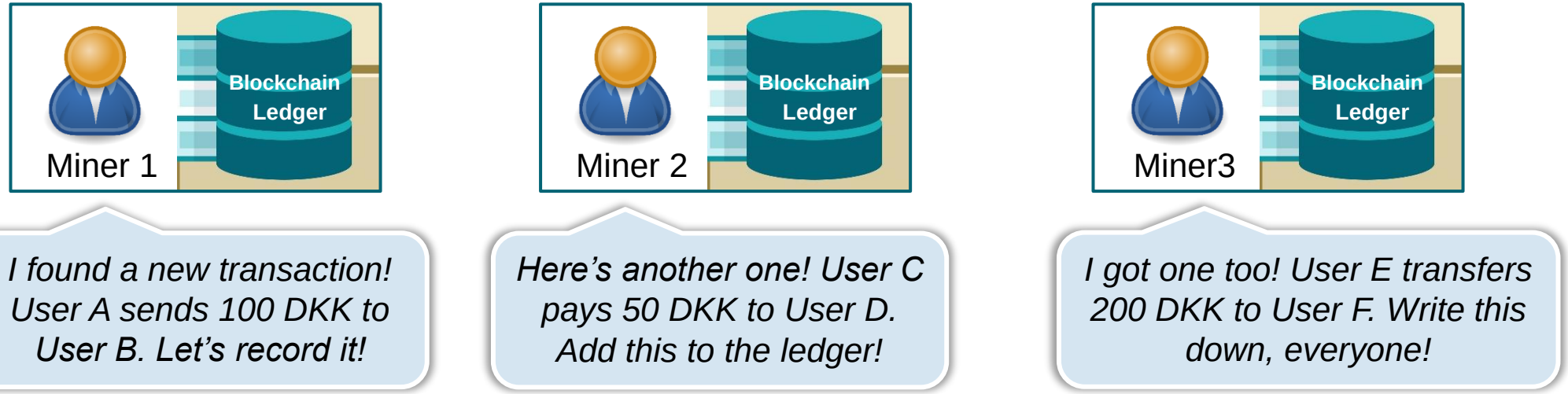
- Bitcoin system works like MobilePay or PayPal, i.e., you can use it to send and receive value.
  - The Bitcoin system uses a blockchain, a shared database run by many participants, called *miners*.
  - All miners in the Bitcoin system keep a copy of this data.
  - Anyone can join the system as a miner and help maintain the ledger.
  - Example:



✓ *Cross-checking* ✓ *Mutual supervision* ✓ *Decentralized control*

Bitcoin

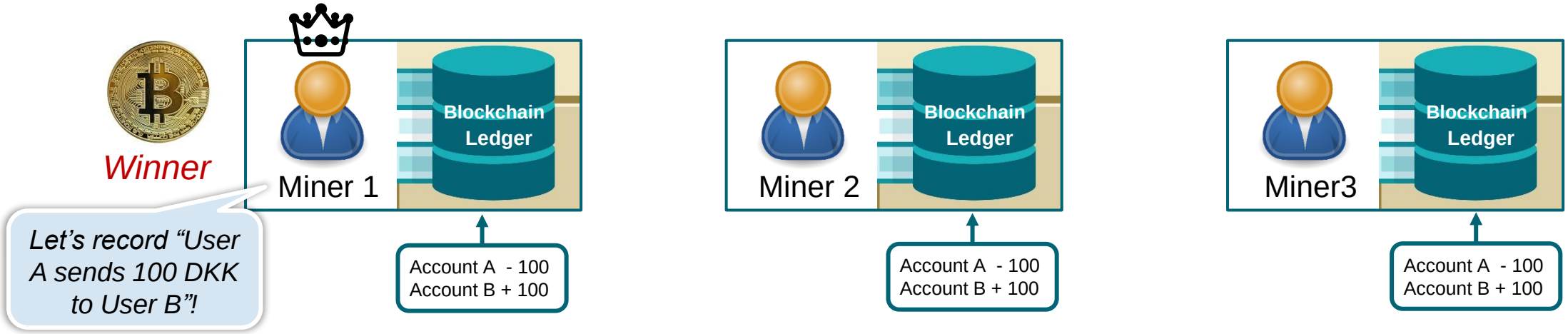
- Bitcoin (BTC) is the reward that powers the Bitcoin system.
  - The system needs many miners to check and record transactions.
    - Thousands of miners keep copies of the Bitcoin database.
    - If everyone tried to update it at the same time, records would clash and no one could tell which version was correct.



How to keep one shared ledger when everyone wants to record at the same time?

Bitcoin

- Bitcoin (BTC) is the reward that powers the Bitcoin system.
  - Bitcoin system allows only one update at a time:
    - Miners run computers and compete by solving a math puzzle.
    - The winner gets the right to update the shared ledger.
    - After the update, the winner earns new bitcoin as a reward.
    - This process keeps all copies the same and lets the system stay fair and transparent.

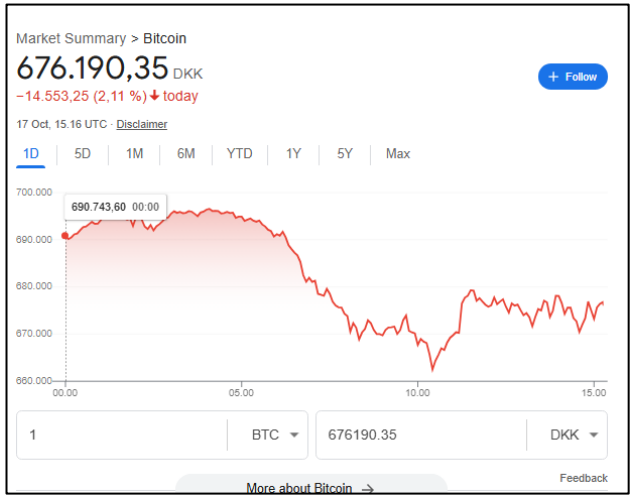


Bitcoin

- A fun fact about Bitcoin (BTC)
  - Anyone can mine new bitcoin — *even you*.
  - Thousands of people and companies run computers to compete for new coins.
  - Mining means solving a math puzzle faster than everyone else.
  - The more computing power you have, the higher your chance to win.
  - Because of this reward system, Bitcoin mining has grown into a global industry.

Bitcoin mining farm

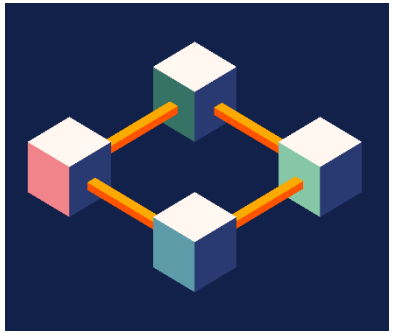
- As of 17 Oct 2025, 1 Bitcoin ≈ 676,000 DKK.



From Bitcoin to Blockchain Databases

- The Bitcoin system runs reliably without a central server.
- Its core technology, the blockchain, stores data that everyone can see and verify.
- These features make blockchain useful beyond digital money.

- Many people began to use it as a database system to store and check shared data.
- A blockchain database helps prevent tampering, data loss, and hidden updates that happen in centralized systems.

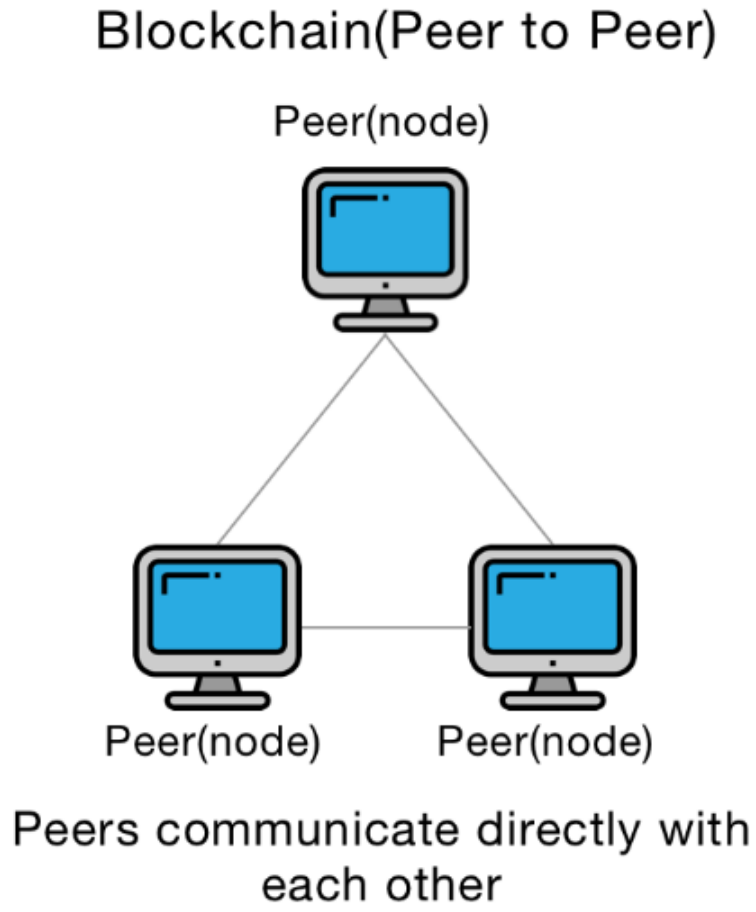


Agenda

- History and Motivation
- Basic Concepts
- Blockchain Architecture
  - Network Layer
  - Consensus Layer
  - Data Layer
  - Smart Contract Layer
  - Application Layer
- Blockchain Applications

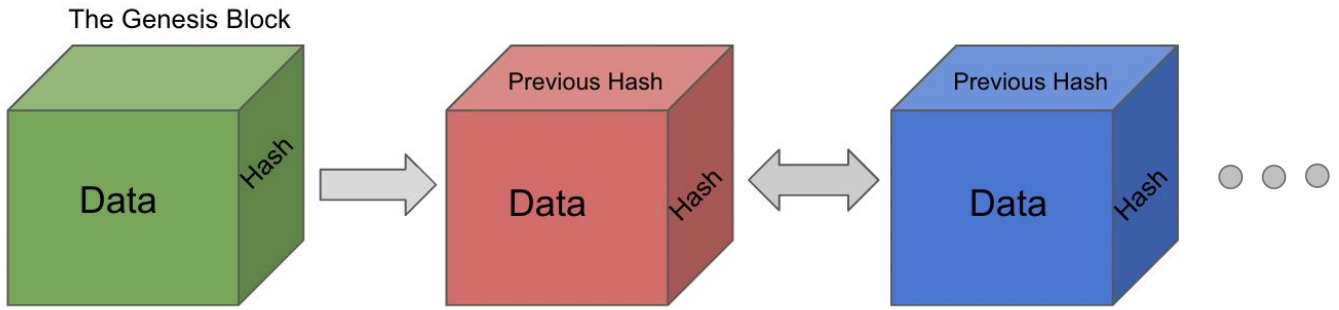
What Is a Blockchain

- A blockchain (also called a distributed ledger) is a distributed data management system maintained by multiple independent participants (also called peer, node, or miner).
  - Each participant keeps a replica of the data, often referred to as a ledger.
  - All participants are connected in a peer-to-peer (P2P) network and exchange information directly.
  - Participants may not trust each other, so the system relies on a consensus mechanism to decide what data is valid and shared by everyone.



The Structure of the Ledger

- A ledger is an ordered, append-only chain of blocks that grows over time.
  - Each block contains a group of new data entries, such as transactions or records.
  - The blocks are linked in sequence, each one referencing the previous block through a cryptographic hash — a unique digital fingerprint of that block.
  - Because each link depends on the exact content of the previous block, changing any block would also change its hash, breaking all the links that follow. As a result, once a block is added, it cannot be removed or modified without being detected.
  - The structure guarantees that the ledger is:
    - Sequential: all records have a fixed order.
    - Immutable: past records cannot be modified.
    - Transparent: everyone can verify the full history.



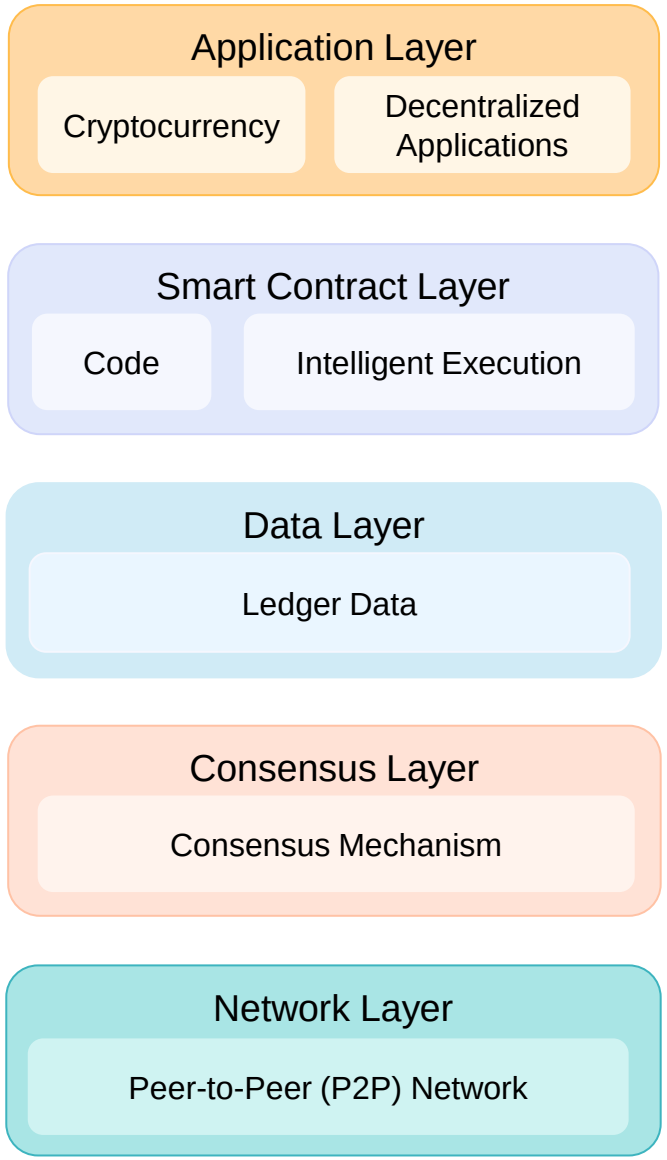
Types of Blockchains

Type	Access and Control	Characteristics	Typical Use Cases
Public Blockchain	• Open to everyone. • Anyone can read, write, and participate in consensus.	• Fully decentralized. • Transparent and censorship-resistant. • Low performance. • Limited privacy.	• Cryptocurrencies (e.g., Bitcoin). • Public record systems. • Open data sharing.
Consortium Blockchain	• Controlled by a group of pre-selected organizations. • Only approved participants can write or validate data.	• Partially decentralized. • Higher performance and better privacy. • Shared governance among organizations.	• Inter-bank payment systems. • Supply-chain management. • Multi-party collaboration.
Private Blockchain	• Managed by a single organization. • Access and validation restricted to internal nodes.	• Centralized control. • High performance and easy management. • Low level of decentralization.	• Enterprise data auditing. • Internal asset tracking. • Permissioned record keeping.

Agenda

- History and Motivation
- Basic Concepts
- Blockchain Architecture
  - Network Layer
  - Consensus Layer
  - Data Layer
  - Smart Contract Layer
  - Application Layer
- Blockchain Applications

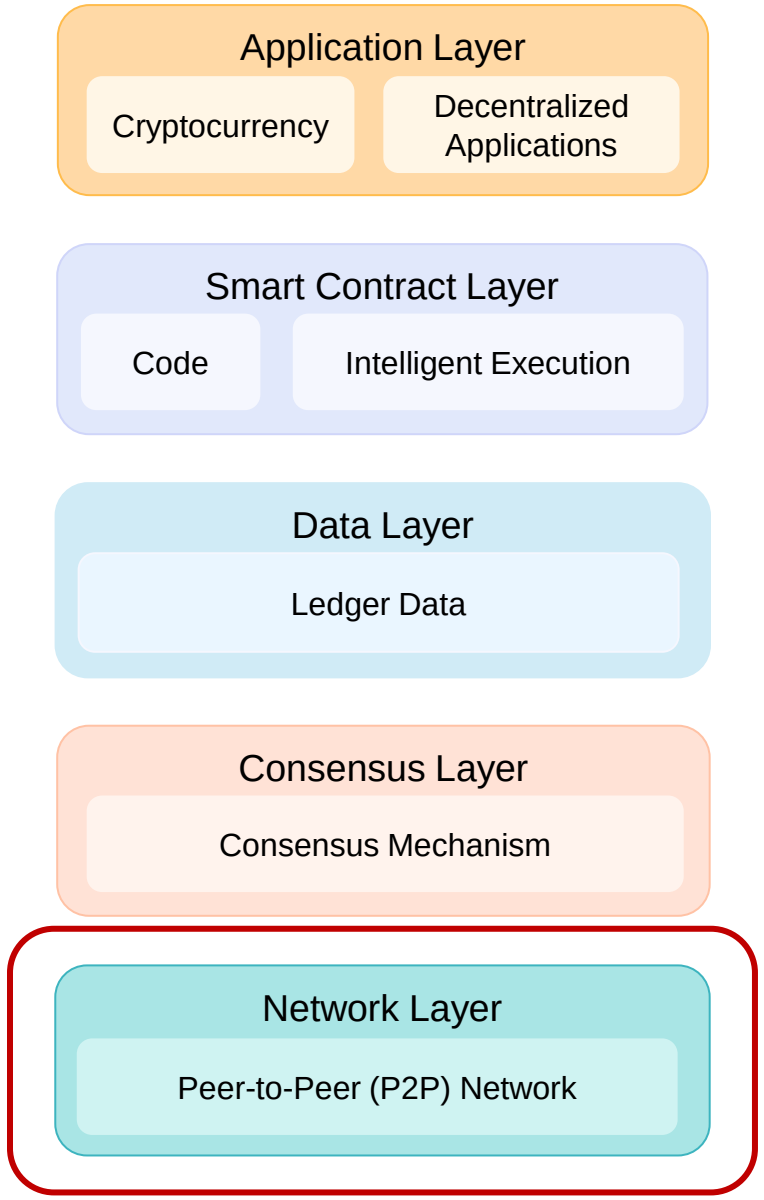
Blockchain Architecture



- A blockchain has multiple layers. It combines communication, data storage, agreement, and application logic in one system.

1. Network Layer
   - It connects all participants and enables data exchange.
2. Consensus Layer
   - It ensures all participants agree on the same data.
3. Data Layer
   - It stores and links data in an ordered, tamper-resistant form.
4. Smart Contract Layer
   - It supports automatic execution of predefined rules.
5. Application Layer
   - It provides user-facing services such as cryptocurrency or decentralized applications.

Network Layer

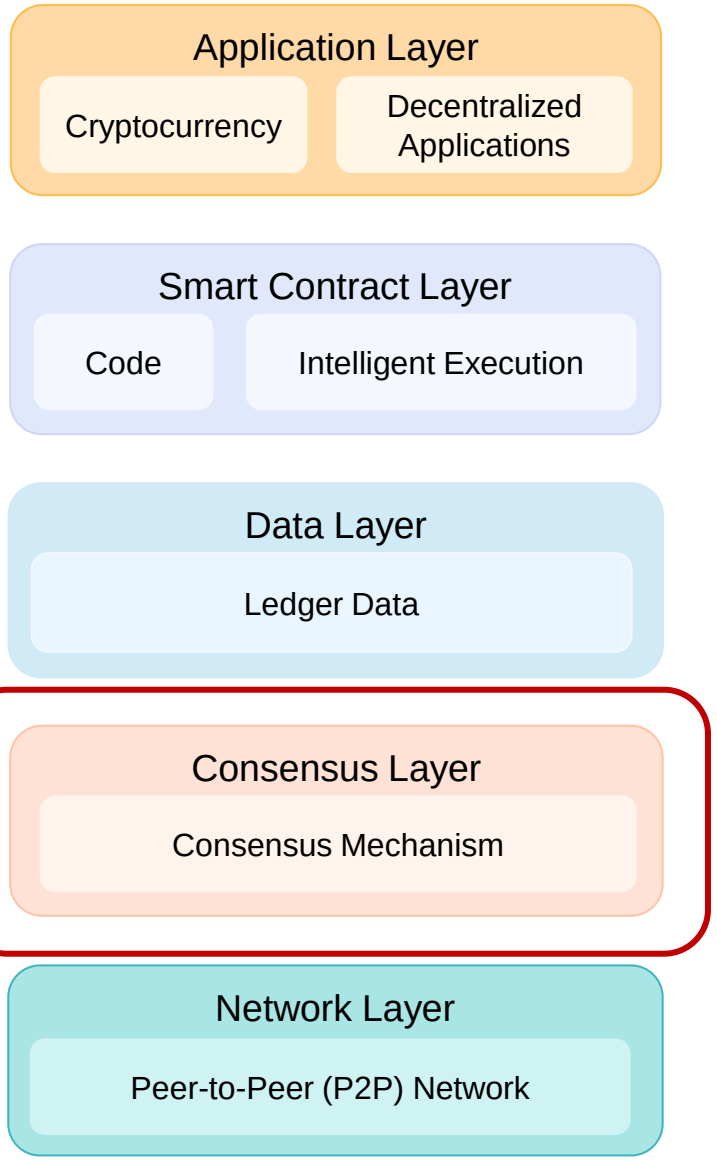


- In network layer, a peer-to-peer (P2P) network is used so that all nodes can
  - Broadcast and validate transactions
  - Propagate new blocks
  - Maintain data consistency without a central server
- A P2P network is a distributed communication model where each node (peer) can act as both a client and a server.
  - Decentralization: No central authority or single control point; all peers are equal.
  - Direct communication: Peers connect and exchange data directly with one another.
  - Scalability: The network can grow and handle more activity as new peers join.
  - Fault tolerance: The system continues to operate even if some nodes fail.

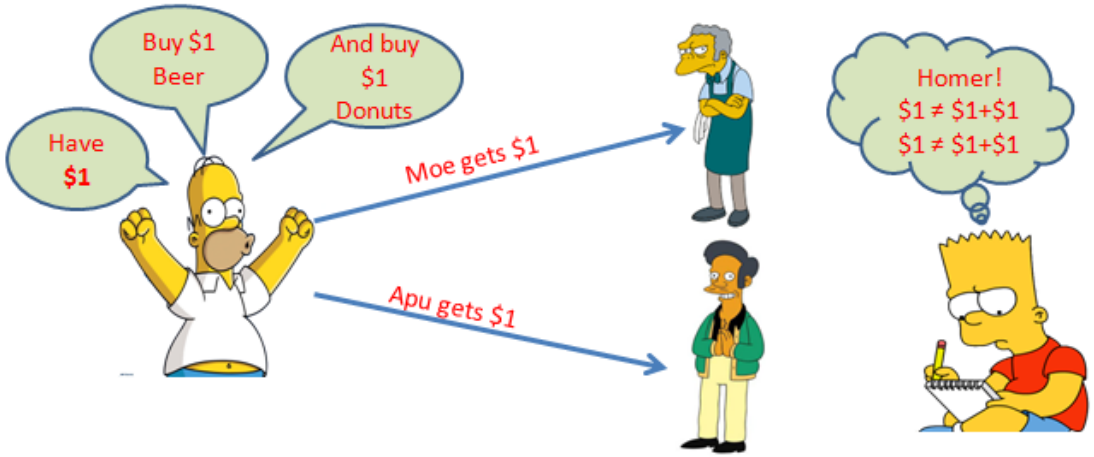
Agenda

- History and Motivation
- Basic Concepts
- Blockchain Architecture
  - Network Layer
  - Consensus Layer
  - Data Layer
  - Smart Contract Layer
  - Application Layer
- Blockchain Applications

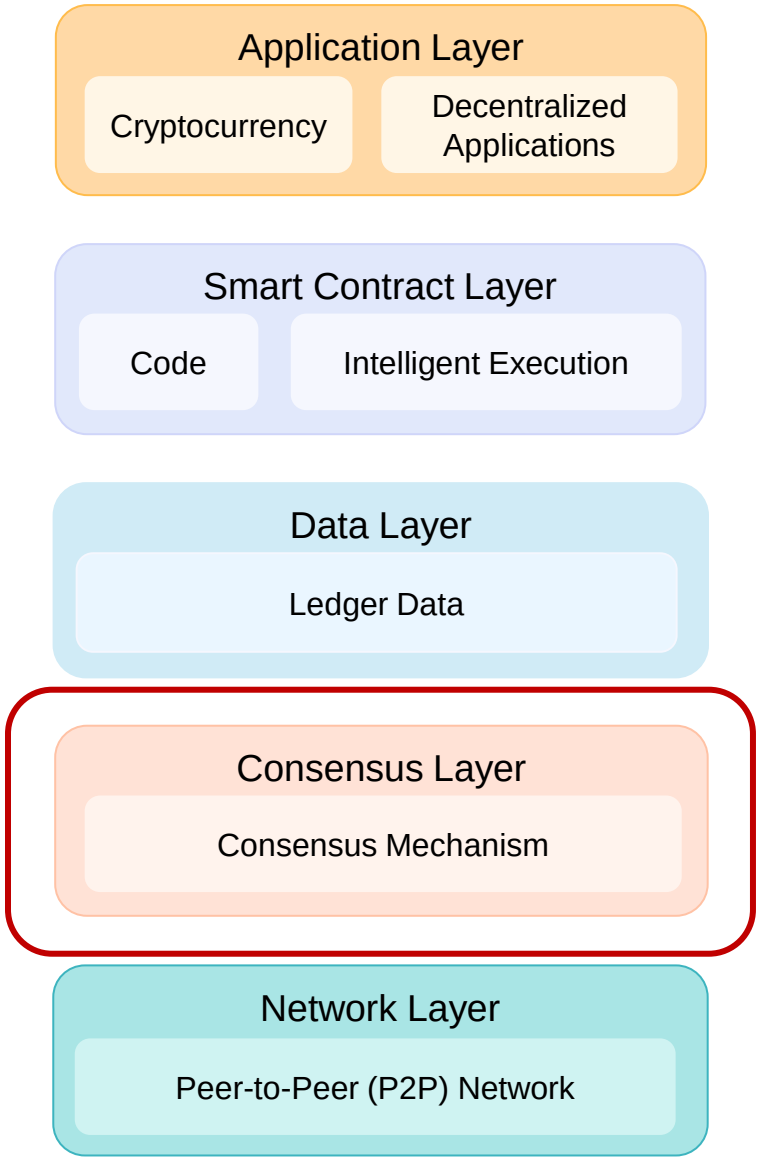
Consensus Layer



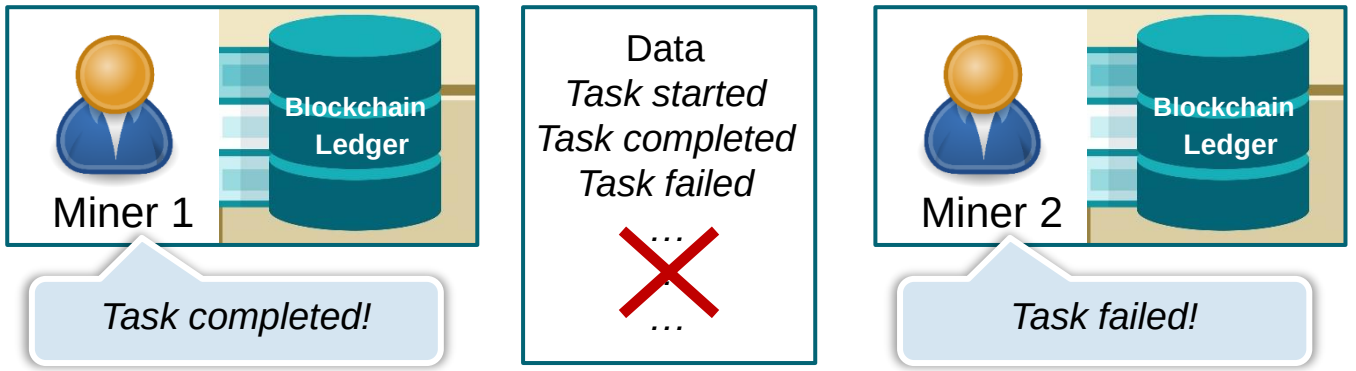
- The consensus layer provides a consensus mechanism. It is the protocol that blockchain nodes use to agree on the current state of the ledger.
- Ensure that all nodes share one consistent version of the truth by preventing double-spending and tolerating Byzantine faults.
  - Double-spending problem
    - Homer has only 1 dollar.
    - He tries to buy beer from Moe and buy donuts from Apu using the same dollar.
    - Both Moe and Apu believe they are getting paid, but in reality only one of them can receive the real dollar
    - This creates two conflicting transactions using the same money.



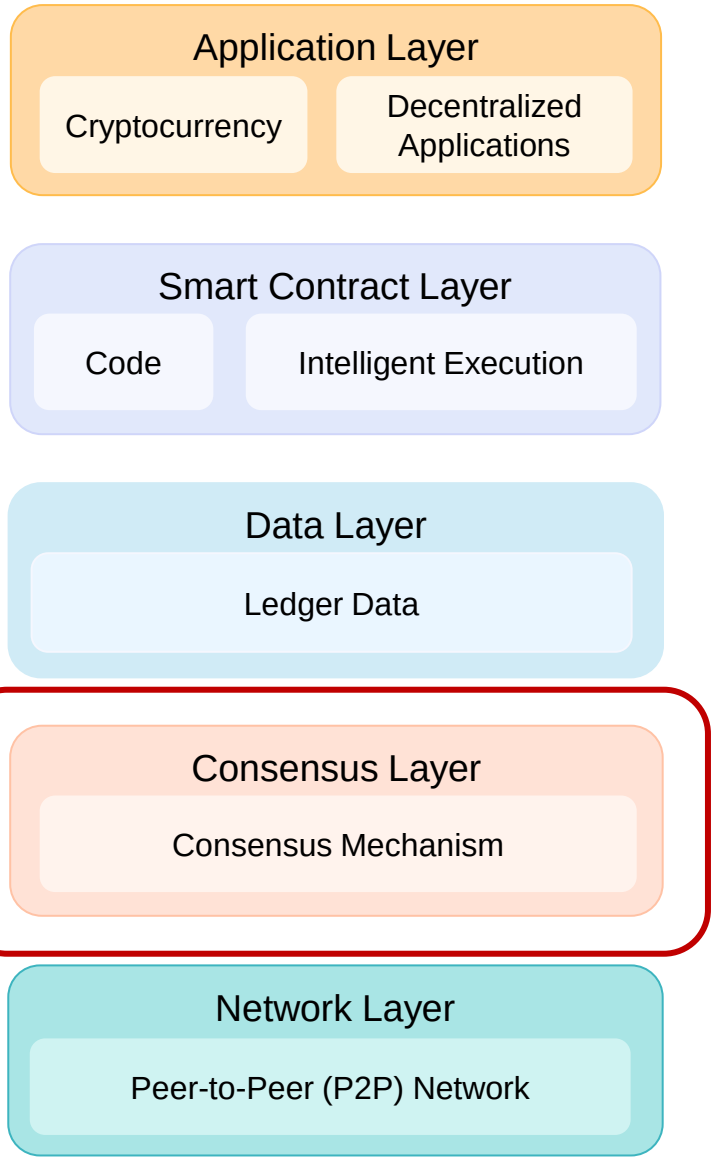
Consensus Layer



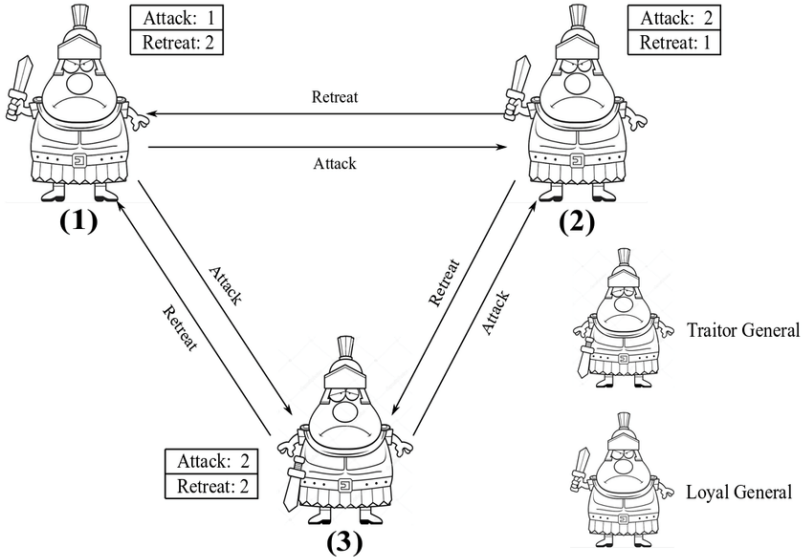
- The consensus layer provides a consensus mechanism. It is the protocol that blockchain nodes use to agree on the current state of the ledger.
- Ensure that all nodes share one consistent version of the truth by preventing double-spending and tolerating Byzantine faults.
  - Double-spending problem in blockchain
    - Double-spending refers to writing the same piece of data more than once or creating conflicting records.
    - For example, two nodes may try to record different results for the same transaction ID at the same time.
    - One node records “Task completed,” while another records “Task failed.”
    - If both records were accepted, the ledger would contain two conflicting versions of the same event.



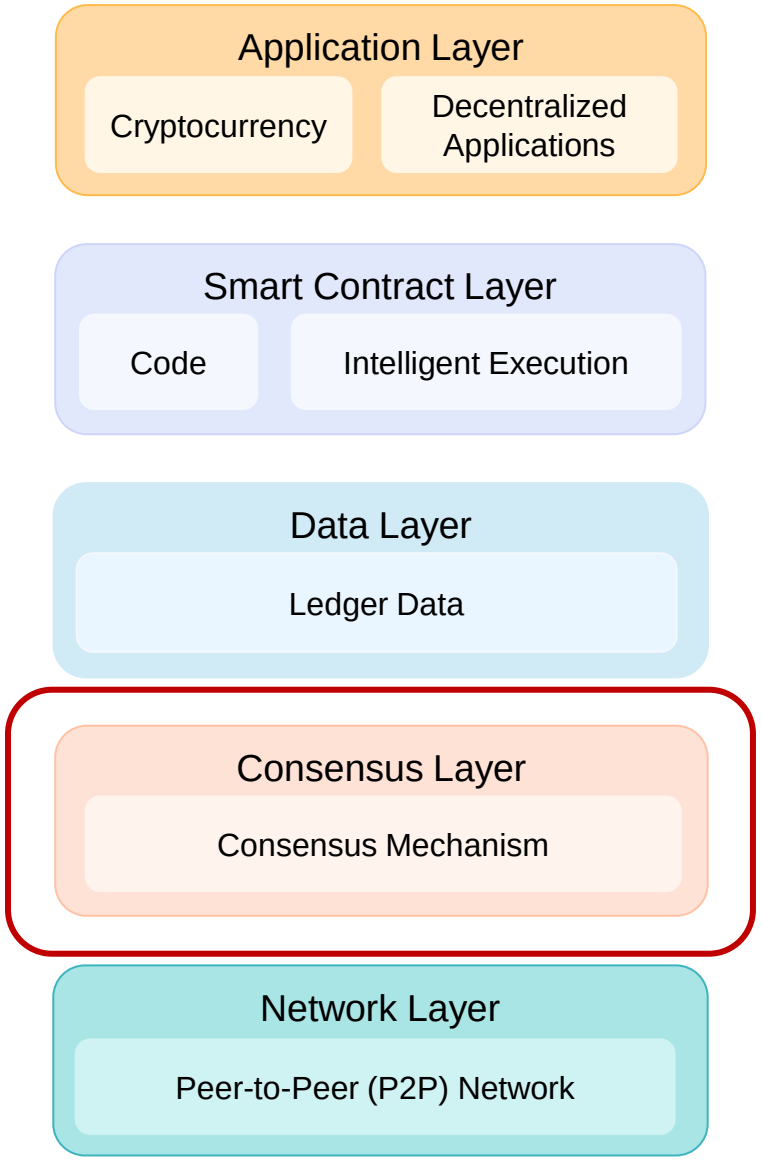
Consensus Layer



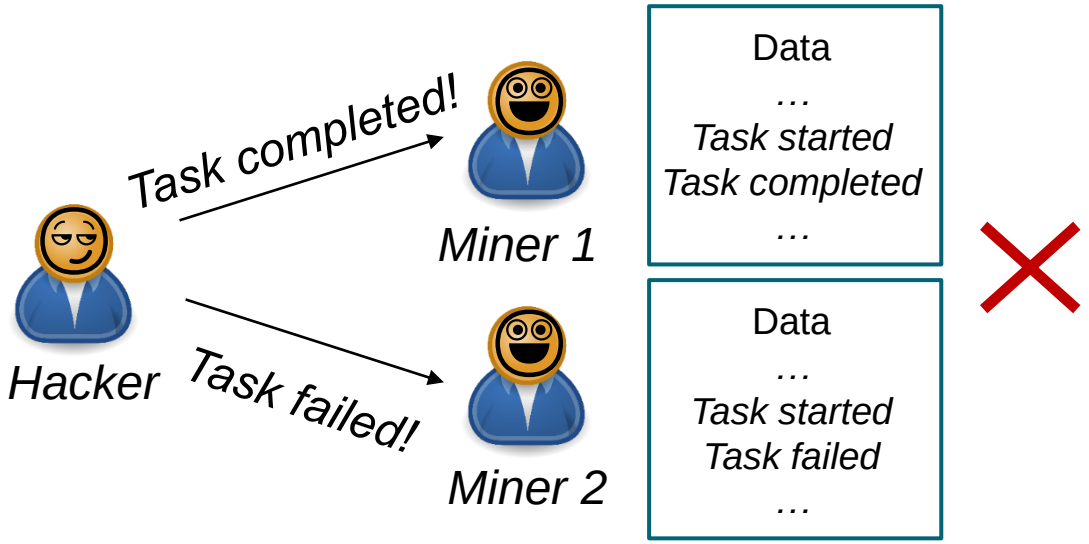
- The consensus layer provides a consensus mechanism. It is the protocol that blockchain nodes use to agree on the current state of the ledger.
- Ensure that all nodes share one consistent version of the truth by preventing double-spending and tolerating Byzantine faults.
  - Byzantine faults
    - Each general must decide whether to attack or retreat.
    - General 2 is a traitor. He tells General 1 to retreat and tells General 3 to attack.
    - As a result, Generals 1 and 3 receive conflicting orders and make different decisions.
    - The army loses because they cannot act in a coordinated way.



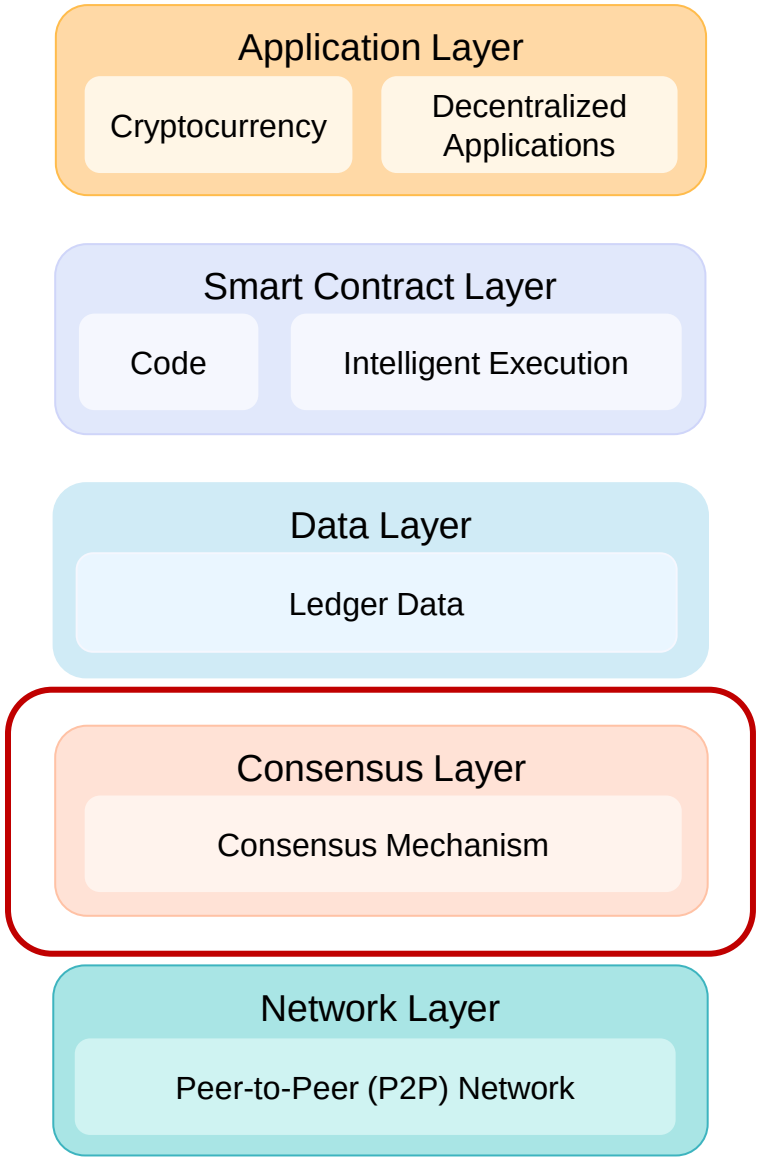
Consensus Layer



- The consensus layer provides a consensus mechanism. It is the protocol that blockchain nodes use to agree on the current state of the ledger.
- Ensure that all nodes share one consistent version of the truth by preventing double-spending and tolerating Byzantine faults.
  - Byzantine faults in blockchain
    - A Byzantine fault occurs when some nodes act incorrectly or dishonestly, sending false data or refusing to cooperate.
    - Example: A faulty node may send different versions of a block to different peers. Different nodes could end up with conflicting copies of the blockchain.



Consensus Layer



- The consensus layer achieves agreement among nodes through a consensus mechanism.
- This mechanisms ensure that all nodes share one consistent version of the truth by preventing double-spending and tolerating Byzantine faults.
- Examples of Common Consensus Mechanisms
  - Proof of Work (PoW)
    - Used by Bitcoin
    - Reference: https://bitcoin.org/bitcoin.pdf
  - Proof of Stake (PoS)
    - Used by Ethereum
    - Reference: https://ethereum.org/en/developers/docs/consensus-mechanisms/pos/
  - Practical Byzantine Fault Tolerance (PBFT)
    - Used by Hyperledger Fabric
    - Reference: https://hyperledger-fabric.readthedocs.io/en/latest/consensus.html

Consensus Mechanism: PoW

- Proof of Work is a consensus mechanism that uses computational effort to decide which node can add the next block to the blockchain.

The PoW process generally follows these steps:

1. Transaction collection
   - Nodes collect pending transactions from the network into a candidate block.
2. Block header creation
   - The block header includes previous block hash, Merkle root of transactions, and timestamp.

Consensus Mechanism: PoW

- Proof of Work is a consensus mechanism that uses computational effort to decide which node can add the next block to the blockchain.

The PoW process generally follows these steps:

3. Hash computation (mining)
   - Miners repeatedly change the nonce and hash the block header using SHA-256.
   - The goal: find a hash value lower than the target difficulty.
   - Thousands of miners compete simultaneously.

Consensus Mechanism: PoW

- Proof of Work is a consensus mechanism that uses computational effort to decide which node can add the next block to the blockchain.

The PoW process generally follows these steps:

4. Broadcast and verification by other nodes
   - The first one to find a valid hash broadcasts the new block to the network.
   - Other nodes verify that the hash is valid and that all transactions follow protocol rules.
   - If valid, they accept the block and add it to their local blockchain copy.
5. Reward distribution
   - The successful miner receives a block reward (newly minted coins) plus transaction fees.

Consensus Mechanism: PoS

- Proof of Stake is a consensus mechanism where nodes are selected to create new blocks based on the amount of stake they hold.

The PoS process generally follows these steps:

1. Validator Selection
   - Participants lock up a certain amount of cryptocurrency as their stake.
   - The protocol randomly selects one or more validators to propose the next block, with higher stakes increasing the chance of selection.

Consensus Mechanism: PoS

- Proof of Stake is a consensus mechanism where nodes are selected to create new blocks based on the amount of stake they hold.

The PoS process generally follows these steps:

2. Block header creation
   - The selected validator creates and proposes a new block containing recent transactions.
3. Validation and Voting
   - Other validators verify the block's correctness and vote on its validity.
   - Once enough votes (a quorum) are collected, the block is considered finalized.

Consensus Mechanism: PoS

- Proof of Stake is a consensus mechanism where nodes are selected to create new blocks based on the amount of stake they hold.

The PoS process generally follows these steps:

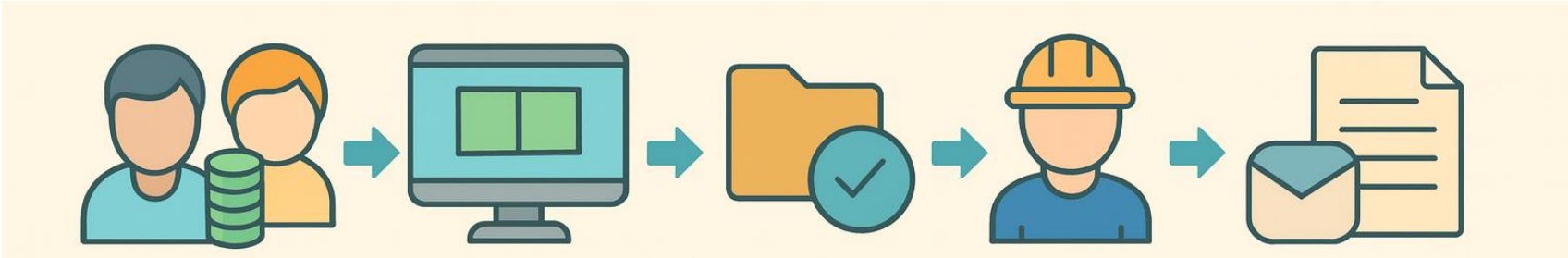
4. Reward and Penalty
   - Honest validators receive rewards (transaction fees or new tokens).
   - Dishonest or inactive validators lose part of their stake (slashing).
5. Finalization
   - The validated block is added to the blockchain, updating the shared ledger across all nodes.

Consensus Mechanism: PBFT

- Byzantine fault tolerance (BFT)
  - BFT ensures correct operation even if some nodes behave maliciously or unpredictably.
  - It originates from the Byzantine Generals Problem, where some participants may lie or fail, but the group must still reach agreement.
  - Allow the system to reach consensus as long as fewer than one-third of the nodes are faulty or malicious.
- Core idea: majority agreement
  - The system follows the decision supported by the majority of honest nodes.
  - If there are $3f + 1$ nodes, the system can tolerate up to f faulty nodes and still reach the same result.
  - Example: Assume there are four miners A, B, C, and D. Here f=1, meaning the system can handle one faulty node.
    - If miner D reports a transaction is valid when it is not. Miners A, B, and C compare their results and agree that the transaction is invalid. Because most nodes report the same result, the system accepts the majority view and rejects D's false message (*tolerates up to 1 faulty node*).
    - If two miners, C and D, both send false information, only A and B remain honest. The honest nodes are no longer the majority, and the system cannot determine which result is correct (*more than 1 faulty node, cannot tolerate*).

Consensus Mechanism: PBFT

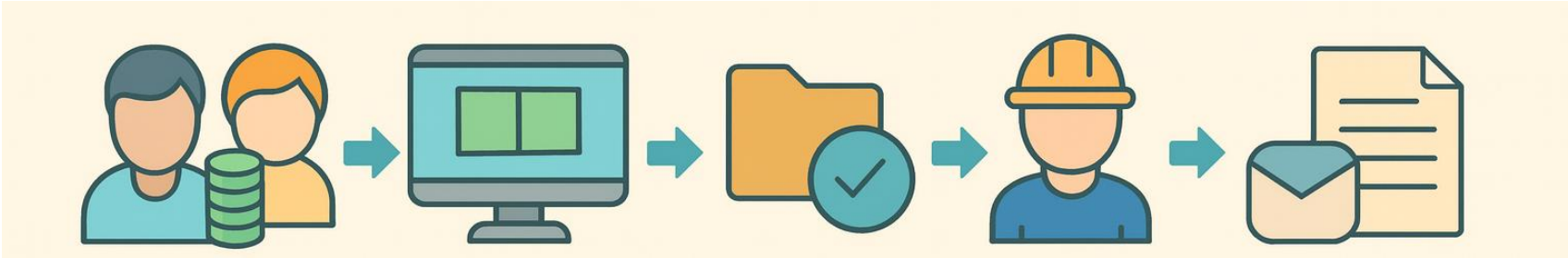
The PBFT process generally follows these steps:



1. Node Roles
   - One node is designated as the primary (leader), and others act as replicas (backups).
   - The leader coordinates the consensus for each round.
2. Block Header Creation
   - The primary node receives a client's request and broadcasts a pre-prepare message to all replicas, proposing the next block or transaction order.
3. Prepare Phase
   - Each replica verifies the proposal and broadcasts a prepare message to other replicas.
   - Nodes collect prepare messages from others to ensure most nodes agree on the same proposal.

Consensus Mechanism: PBFT

The PBFT process generally follows these steps:



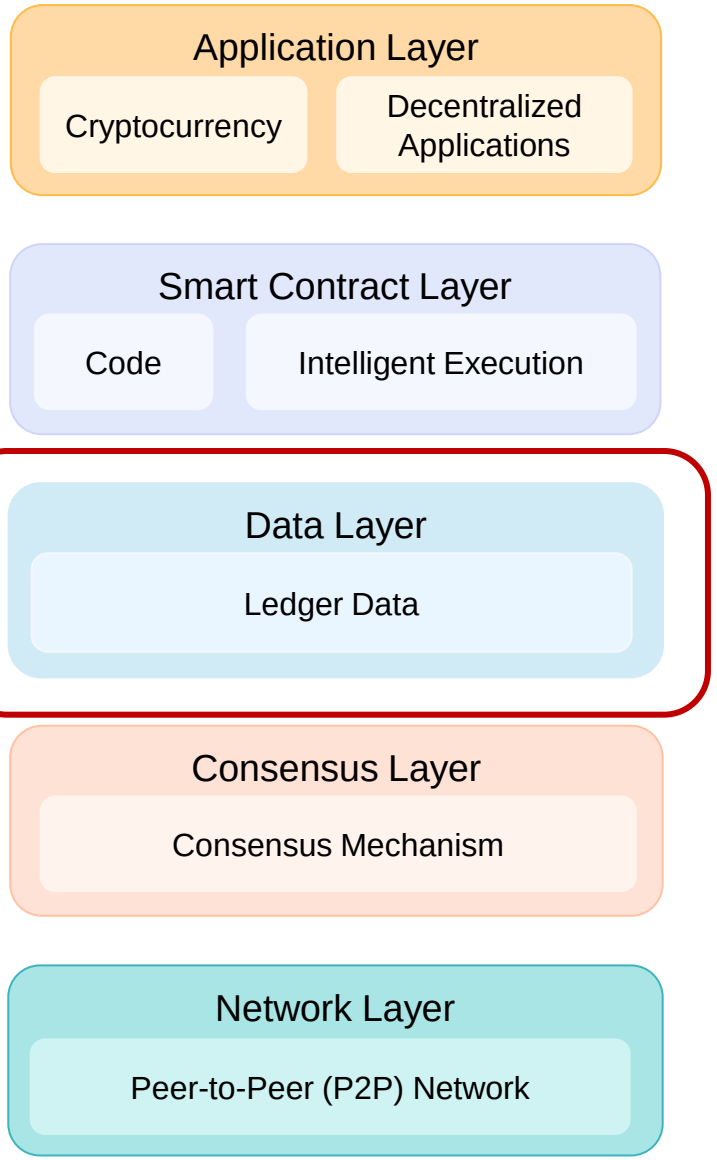
4. Commit Phase
   - Once a node receives enough prepare messages, it broadcasts a commit message.
   - When a node receives enough commit messages (typically from at least $2f + 1$ nodes, where f is the number of faulty nodes tolerated), it executes the transaction and updates its ledger.

5. Reply to Client
   - Each node sends the result back to the client, which accepts the result after receiving consistent replies from the majority.

Agenda

- History and Motivation
- Basic Concepts
- Blockchain Architecture
  - Network Layer
  - Consensus Layer
  - Data Layer
  - Smart Contract Layer
  - Application Layer
- Blockchain Applications

Data Layer

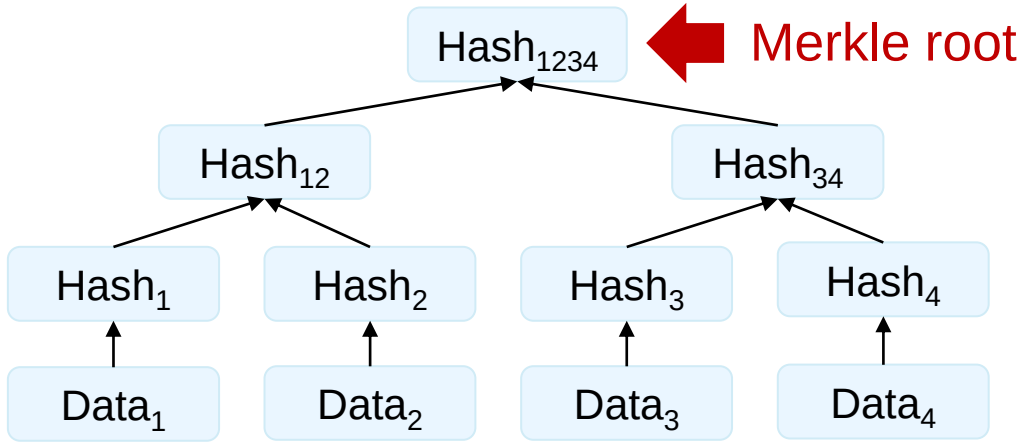


- The data layer stores the chain's data history and ledger data in a structured and immutable way.
- Data is organized into blocks, and each block is cryptographically linked to the previous one, forming a continuous chain.
- Each block has two main parts:
  - Block header
    - *Hash of the previous block* records the unique fingerprint of the block before it, ensuring every block is connected and any change can be detected.
    - *Timestamp* marks the exact time the block was created, keeping the order of transactions consistent.
    - *Merkle root* summarizes all transactions in the block through hashing, allowing quick verification of any individual transaction.
  - Block body
    - Contains the complete list of validated data that are permanently recorded in the ledger.

Data Layer: Merkle Tree

- Merkle tree is used to summarize and verify all data stored in a block efficiently.
  - Each leaf node represents the hash of a single piece of data, and every non-leaf node stores the hash of its two child nodes.
  - The hash function used is typically SHA-256 (see https://en.wikipedia.org/wiki/SHA-2), which converts any input data into a fixed 256-bit value.
    - Example: SHA-256*(Hello)* = 185f8db32271fe25f561a6fc938b2e264306ec304eda518007d1764826381969

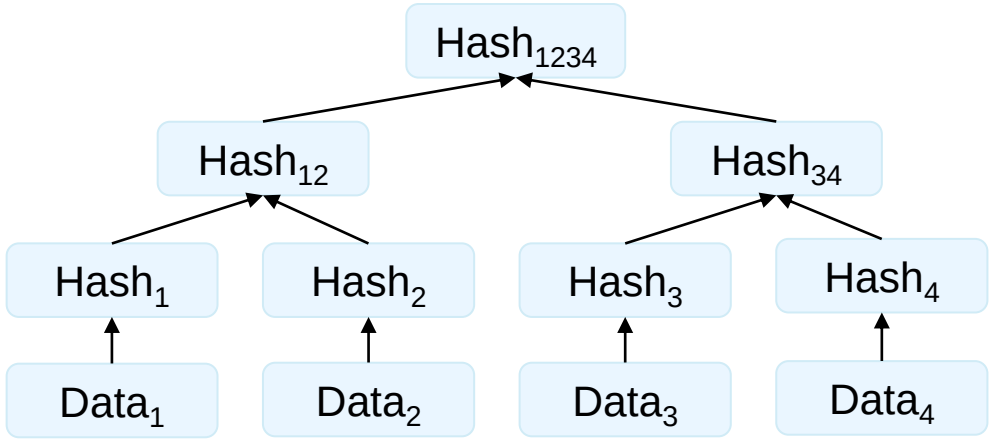
      (This is a 64-character hexadecimal value, representing a 256-bit hash.)
  - This process continues upward until one final hash, called the Merkle root, is generated.



Data Layer: Merkle Tree

- A hash function in Merkle tree converts any input data into a fixed-size value called a hash value or digest.

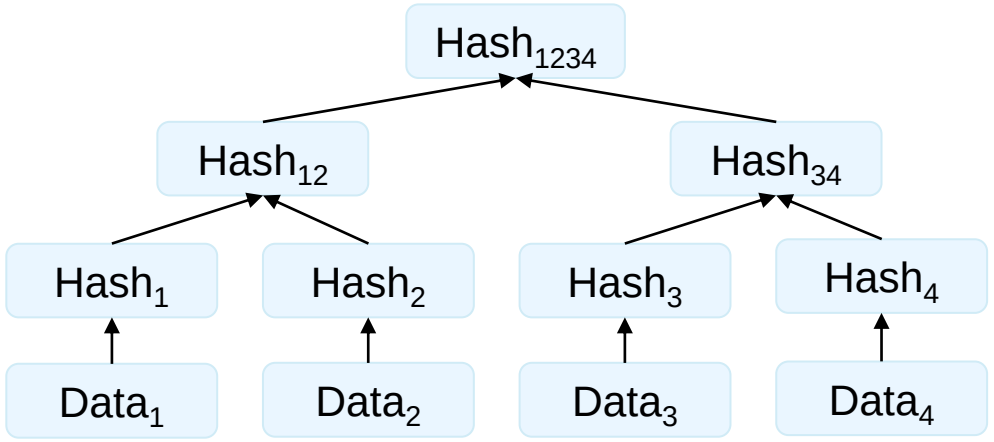
- What it does
  - Identifies data uniquely with a short digital fingerprint.
  - Detects any change in the data instantly.
  - Links blocks together securely in the blockchain.



- Key Properties
  - Deterministic: Same input always gives the same hash.
  - Irreversible: Cannot recover the original data from the hash.
  - Collision-resistant: Hard to find two different inputs with the same hash.
  - Avalanche effect: Small input changes cause completely different hashes.

Data Layer: Merkle Tree

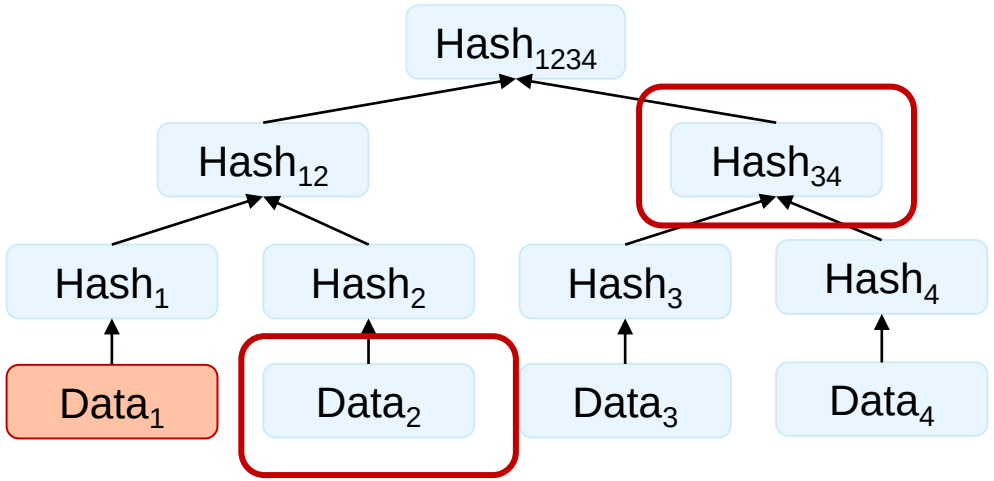
- Checking data integrity with the Merkle root
  - To check whether a block's data has been changed, recompute the Merkle root from all data in the block and compare it with the Merkle root stored in the block header.
  - If the two values differ, it means the block's content has been modified.
  - Any modification to the data is transmitted through the tree structure and reflected in the root hash, ensuring that tampering can be detected.
  - Example: Given $Data_1$–$Data_4$ as all data items in a block
    - If $Data_1$ changes, $Hash_1$ will change.
    - The combined hash $Hash_{12}$ (computed from $Hash_1$ and $Hash_2$) will change.
    - This change will propagate upward, causing $Hash_{1234}$ (the Merkle root) to change.



- $Hash_1 = Hash(Data_1)$
- $Hash_{12} = Hash(Hash_1 + Hash_2)$
- Merkle root
  $Hash_{1234} = Hash(Hash_{12} + Hash_{34})$

Data Layer: Merkle Tree

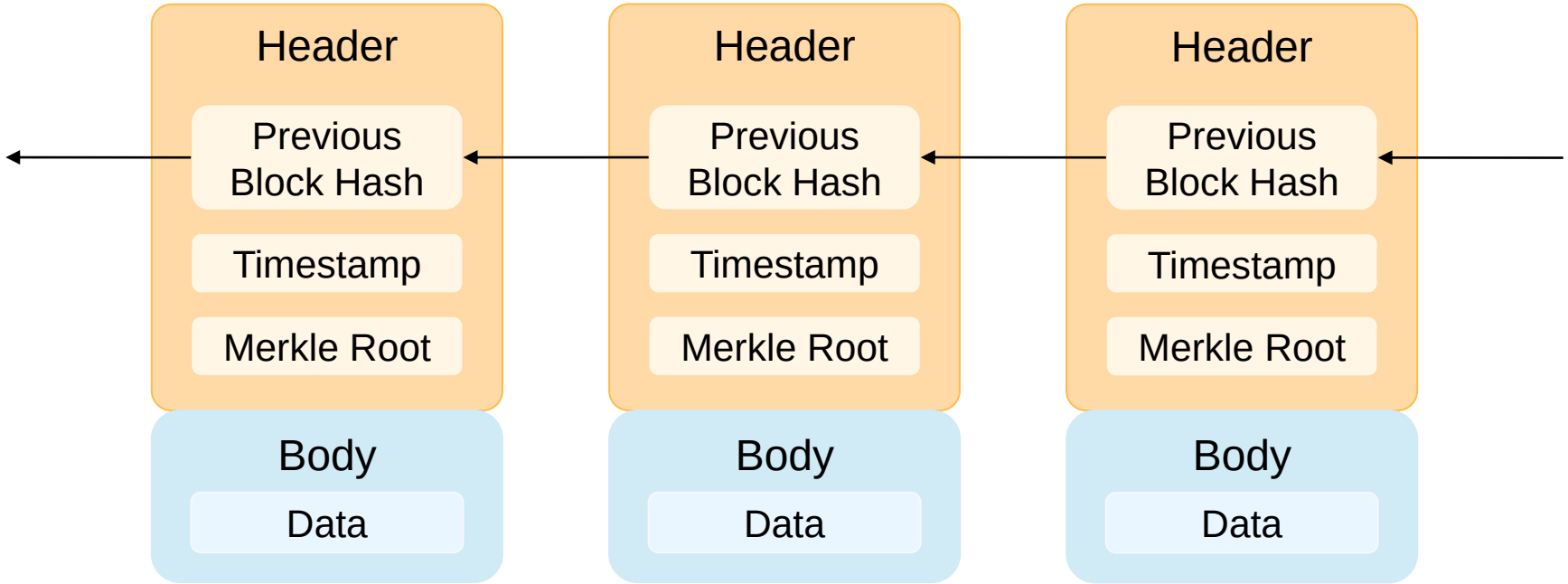
- Verifying data inclusion with the Merkle root
  - To verify that a specific data item exists in a block, compute a hash path from that item up to the Merkle root and compare the result with the Merkle root recorded in the block header.
  - If the two roots are identical, the data is confirmed to be part of the block.
  - Example: Given $Data_1$–$Data_4$ as all data items in a block
    - Verifying $Data_1$ requires only $Data_2$ and $Hash_{34}$, not all data in the block.
    - Use $Data_2$ to compute $Hash_{12}$ and combine it with $Hash_{34}$ to compute the root $Hash_{1234}$.
    - If the computed root matches the Merkle root in the block header, it proves that $Data_1$ is included and unaltered in the block.



- $Hash_1 = Hash(Data_1)$
- $Hash_{12} = Hash(Hash_1 + Hash_2)$
- Merkle root $Hash_{1234} = Hash(Hash_{12} + Hash_{34})$

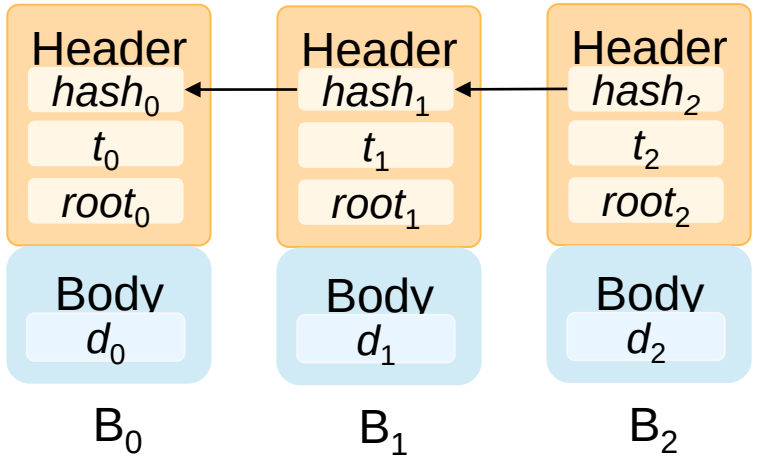
Data Layer: Hash Chain

- Each block header contains the hash of its predecessor, which records the unique fingerprint of the previous block.
- This connection forms a continuous hash-linked chain. If any earlier block is changed, its hash will also change, and all following links will become invalid.
- This structure makes the blockchain tamper-evident and preserves the integrity of the entire chain.



Data Layer: Hash Chain

- Verifying data integrity using the hash chain
  - When verifying the chain, recalculate each block's hash using its timestamp and Merkle root, then compare it with the "previous block hash" stored in the next block.
  - If all hashes match, the chain remains intact.
  - If any mismatch appears, it means an earlier block has been changed, making all following links invalid.
  - Example: Given a blockchain with three blocks B_0, B_1, and B_2
    - To check whether B_2 is correctly linked, recalculate $hash_1$ using B_1 's timestamp and Merkle root.
    - Then use the result to compute the expected $hash_2$
    - Compare this computed $hash_2$ with the "previous block hash" stored in B_2's header.
    - If they match, B_2 is correctly connected to B_1 .
    - If not, it means B_1 or B_2 has been modified.

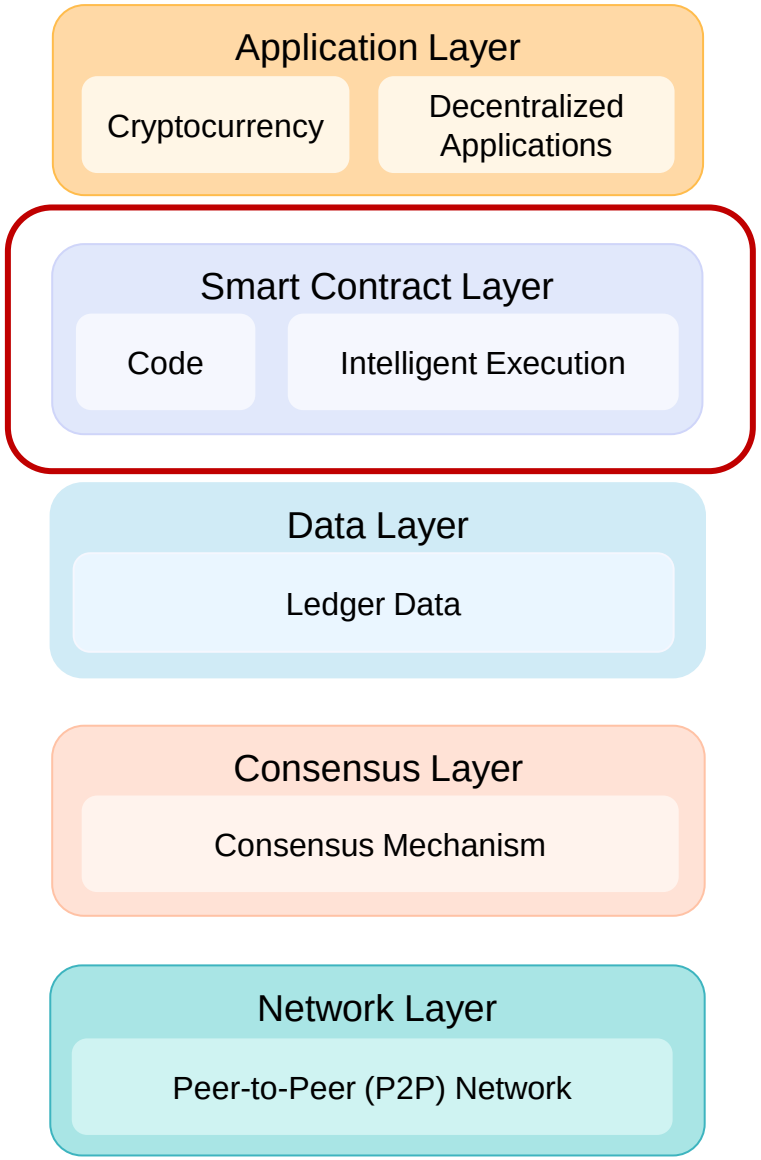


- $hash_0$=*NULL*
- $hash_1$=Hash($hash_0$+t_0+$root_0$)
- $hash_2$=Hash($hash_1$+t_1+$root_1$)

Agenda

- History and Motivation
- Basic Concepts
- Blockchain Architecture
  - Network Layer
  - Consensus Layer
  - Data Layer
  - Smart Contract Layer
  - Application Layer
- Blockchain Applications

Smart Contract Layer

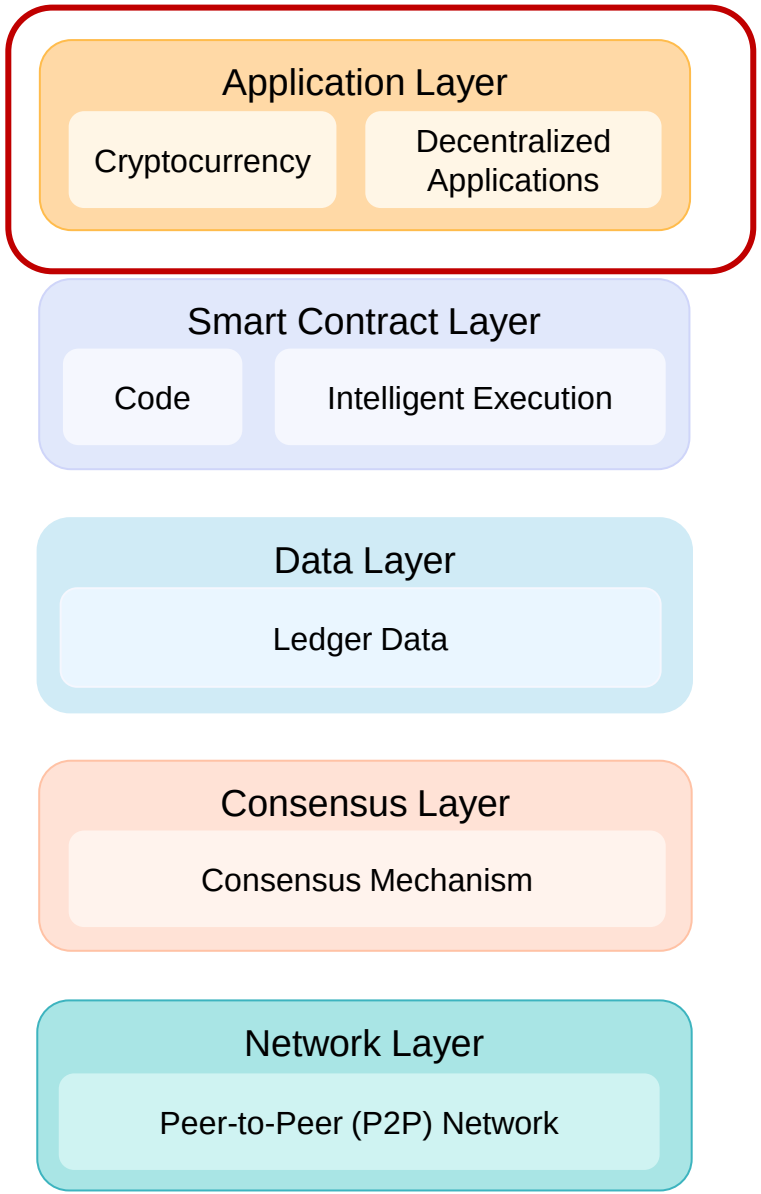


- A smart contract is a self-executing program stored on the blockchain. It automatically enforces rules, checks conditions, and performs predefined actions without intermediaries.
- “Code is law.”
  - Once deployed, the contract runs exactly as programmed and cannot be changed arbitrarily.
- How it works
  - A user sends a request that triggers a function in the contract.
  - All nodes in the network execute the same code and verify the outcome.
  - After verification, the contract’s state is updated on the blockchain.

Agenda

- History and Motivation
- Basic Concepts
- Blockchain Architecture
  - Network Layer
  - Consensus Layer
  - Data Layer
  - Smart Contract Layer
  - Application Layer
- Blockchain Applications

Application Layer



- The application layer provides interfaces and services that allow users, organizations, or other systems to interact with the blockchain network.

- Functions
  - User interaction: applications that read from or write data to the blockchain and present information to users.
  - Business logic: defines and enforces the operational rules for specific use cases through smart contracts.
  - External integration: connects the blockchain with off-chain systems or services such as databases, IoT devices, or APIs.

Strengths

- **Decentralization** --- No central authority; data and control are distributed across all nodes. Reduces single points of failure and improves system resilience.
- **Transparency** --- All transactions are recorded on a public ledger and can be verified by any participant. Builds trust among parties.
- **Immutability** --- Once data is written to the blockchain, it cannot be modified or deleted. Ensures data integrity and tamper resistance.
- **Security** --- Uses cryptographic techniques (hashing, digital signatures) to secure transactions and prevent fraud.
- **Traceability** --- Every transaction can be traced back through the chain, making it ideal for auditing and provenance tracking.
- **Automation via Smart Contracts** --- Business rules can execute automatically without intermediaries, reducing operational costs and human error.

Weakness

- **Scalability** --- Low throughput and long confirmation time compared with centralized databases. Difficult to handle large volumes of transactions.
- **Storage Overhead** --- Every node stores a full copy of the ledger, causing large and ever-growing data size.
- **Limited Privacy** --- Transaction data is transparent; while pseudonymous, it may still expose user activity patterns.
- **Irreversibility** --- Once recorded, transactions cannot be modified or deleted, even if entered by mistake.
- **Complex Maintenance** --- Upgrading protocols or fixing bugs often requires network-wide consensus (hard forks).
- **Regulatory and Legal Uncertainty** --- Many jurisdictions lack clear rules for blockchain-based transactions and smart contracts.

Agenda

- History and Motivation
- Basic Concepts
- Blockchain Architecture
  - Network Layer
  - Consensus Layer
  - Data Layer
  - Smart Contract Layer
  - Application Layer
- Blockchain Applications

Blockchain Applications

- Singapore Exchange (SGX) – Blockchain in Financial Services
  - SGX is a major investment and trading services provider across Asia.
  - They use blockchain technology to build more efficient account and inter-bank payment systems.

Project Ubin: Central Bank Digital Money using Distributed Ledger Technology

What It Is

Project Ubin is a collaborative project with the industry to explore the use of Blockchain and Distributed Ledger Technology (DLT) for clearing and settlement of payments and securities.

The project aims to help MAS and the industry better understand the technology and the potential benefits it may bring through practical experimentation. This is with the eventual goal of developing simpler to use and more efficient alternatives to today's systems based on central bank issued digital tokens.

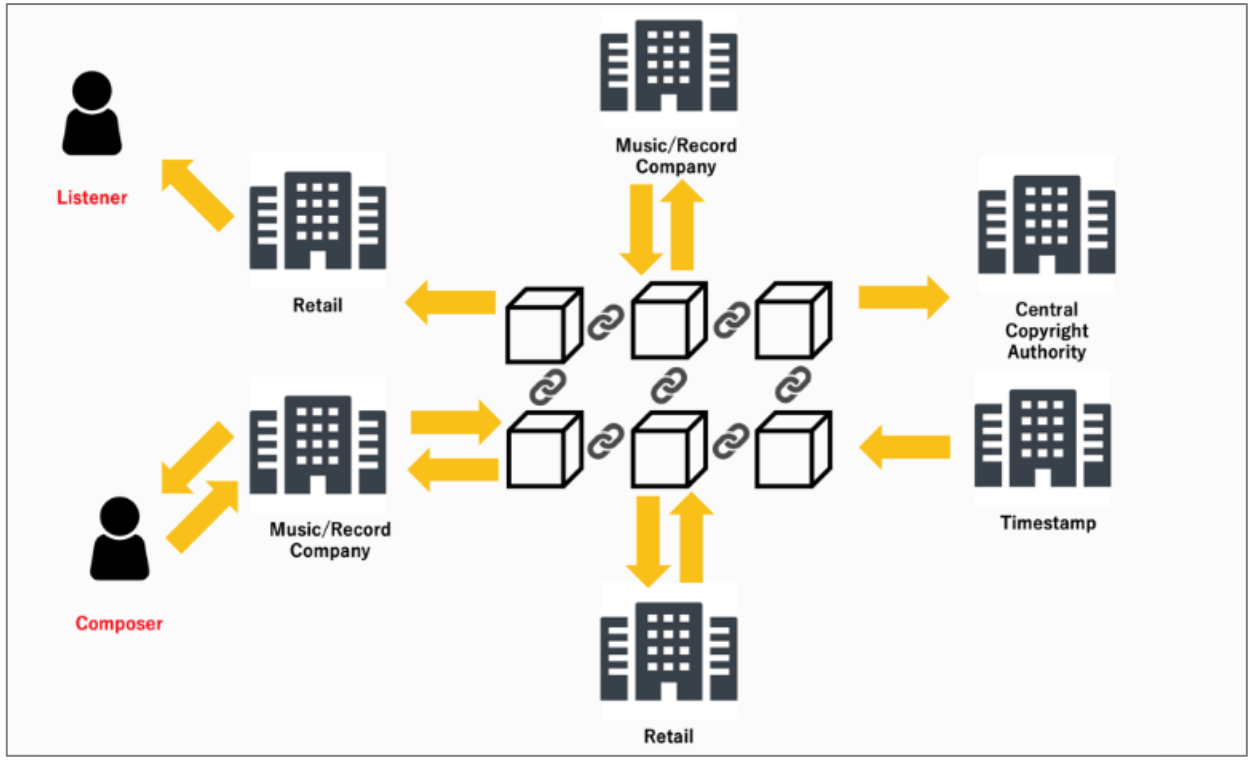
Project Ubin is a multi-year multi-phase project, with each phase aimed at solving the pressing challenges faced by the financial industry and the blockchain ecosystem. The project has successfully concluded in 2020 after five phases, and the publication of 6 project reports.

Project Ubin has informed MAS and the industry on the learnings of DLT and blockchain in the financial industry, and sets the foundation for additional digital asset and digital money initiatives by MAS and the industry such as Ubin+, which explores cross border connectivity with other central banks.

  - Source: https://www.mas.gov.sg/schemes-and-initiatives/project-ubin?utm_source=chatgpt.com

Blockchain Applications

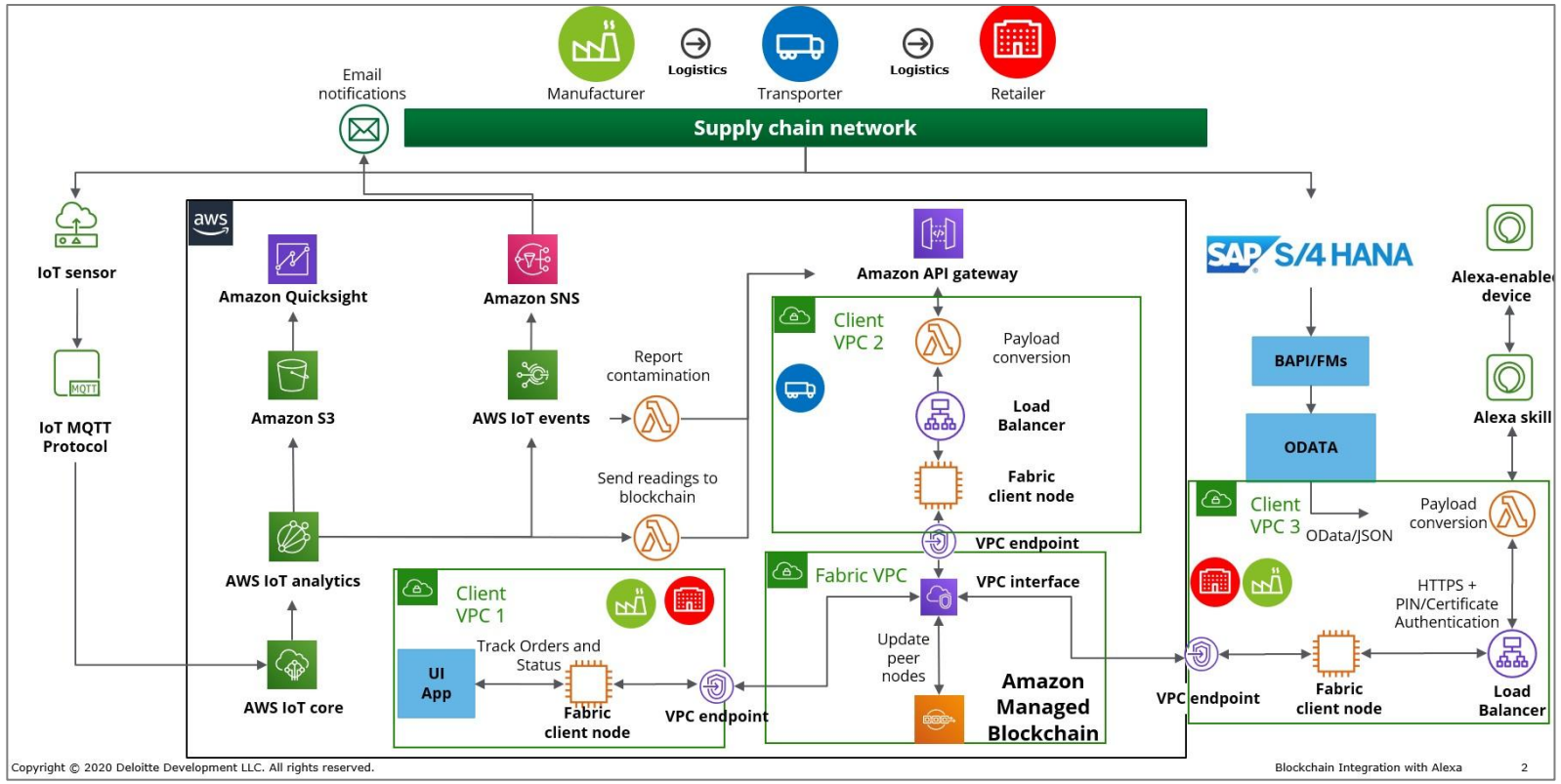
- Sony Music Entertainment Japan – Digital Rights Management (DRM)
  - Sony developed a blockchain-based rights management system for digital content in collaboration with its subsidiaries.



  - Source: https://www.forbes.com/sites/amazonwebservices/2019/11/19/how-sony-is-protecting-rights-of-digital-creators-using-blockchain-on-aws/

Blockchain Applications

- Amazon – Supply Chain Authenticity and Product Tracking
  - Amazon filed patents for a distributed ledger system to verify product authenticity and trace items through its global supply chain.



  - Source: https://aws.amazon.com/cn/blogs/apn/supply-chain-tracking-and-traceability-with-iot-enabled-blockchain-on-aws/

Summary

- History and Motivation
- Basic Concepts
- Blockchain Architecture
  - Network Layer
  - Consensus Layer
  - Data Layer
  - Smart Contract Layer
  - Application Layer
- Blockchain Applications

Reference

- Abraham et al.. Database System Concepts (seventh edition) – chapter 26
- Zhongming Yao. Introduction of Blockchain (in Chinese)